

Lesson 4: JavaScript Algorithms

Intro to Programming — Code the Dream

Session Overview

By the end of this session, you will be able to:

- Define an algorithm and give a real-world example
- Write pseudocode before writing real code
- Solve problems by breaking them into smaller functions
- Pass a function as a callback to another function
- Create and submit a pull request on GitHub

Explore

What Is an Algorithm?

An **algorithm** is a set of steps you write to solve a problem.

You already follow algorithms every day:

- Following a recipe
- Using GPS navigation
- Withdrawing cash from an ATM

In programming, the computer follows your steps exactly — every step must be spelled out.

Algorithms Need Every Step

Making tea — as an algorithm:

1. Check that you have tea, water, a cup, and a kettle
2. If anything is missing — stop
3. Fill the kettle and heat the water
4. Place tea in the cup
5. Pour hot water into the cup
6. Wait 3–5 minutes
7. Remove the tea bag
8. Serve

Skip a step and the result is wrong. Computers work the same way.

Pseudocode: Plan Before You Code

Pseudocode is writing your steps in plain language before writing real code.

It helps you think clearly — even experienced developers do this.

```
function tipCalculator(billTotal, tipPercentage)
  // Check that the inputs are valid
  // Multiply billTotal by tipPercentage to get the tip
  // Add the tip to billTotal to get the final amount
  // Return the final amount
```

Once you write the real code, pseudocode becomes your comments.

Pseudocode Becomes Real Code

```
function tipCalculator(billTotal, tipPercentage) {  
  // Multiply billTotal by tipPercentage to get the tip  
  let tip = billTotal * tipPercentage;  
  // Add the tip to billTotal to get the final amount  
  let total = billTotal + tip;  
  // Return the final amount  
  return total;  
}  
  
console.log(tipCalculator(50, 0.2)); // 60
```

The comments stayed — they explain the code to anyone who reads it later.

Check for Understanding

What is the main purpose of pseudocode?

- A) To run your program faster
- B) To plan your steps in plain language before writing code
- C) To replace `console.log()` in your program
- D) To make your code look professional

Breaking a Big Problem into Small Functions

Some problems have many steps. The trick: **one function per step**.

```
function calculateAverage(scores) { /* step 1 */ }  
function getLetterGrade(average) { /* step 2 */ }  
function passed(letterGrade)      { /* step 3 */ }
```

Then combine them in one final function:

```
function printClassResult(className, student, scores) {  
  let avg    = calculateAverage(scores);  
  let grade  = getLetterGrade(avg);  
  let didPass = passed(grade);  
  // ...  
}
```

What Is a Callback?

A **callback** is a function that you pass to another function.

```
function buttonPushed() {  
  console.log("The button was pushed!");  
}  
  
function simulateButtonPush(callback) {  
  callback();  
}  
  
simulateButtonPush(buttonPushed);  
// "The button was pushed!"
```

We pass `buttonPushed` without parentheses — we are passing the function itself, not calling it.

Why Use Callbacks?

Without callbacks — a separate function for every action:

```
function printDouble(num) { console.log(num * 2); }  
function printTriple(num) { console.log(num * 3); }
```

With a callback — one flexible function:

```
function processNumber(num, action) {  
  action(num);  
}  
  
processNumber(5, (n) => console.log(n * 2)); // 10  
processNumber(5, (n) => console.log(n * 3)); // 15
```

Callbacks make functions **flexible and reusable**.

Check for Understanding

What does this code print?

```
function sayHi() {  
  console.log("Hi there!");  
}  
function run(fn) {  
  fn();  
}  
run(sayHi);
```

- A) Nothing — `sayHi` was never called
- B) An error — you cannot pass a function as an argument
- C) "Hi there!"
- D) `fn`

Git This Week: Pull Requests

A **pull request (PR)** asks to merge your branch into `main`.

This is how real development teams review code:

1. Push your branch to GitHub
2. Open a pull request
3. A reviewer looks at your changes
4. The code gets merged

You will create and submit your first PR this week.

Check for Understanding

You finished your work on `lesson-4` branch. What is the correct order of steps?

- A) Merge → Push → Commit → PR
- B) Commit → Push → Open a PR → Merge after review
- C) Push → Commit → Merge → PR
- D) PR → Push → Commit → Merge

Apply

Core Activity: Let's Code Together

We'll build something step by step — you tell me what to type.

Open your browser console:

- **Chrome / Edge:** Right-click anywhere → Inspect → Console
- **Firefox:** Right-click anywhere → Inspect → Console

Or use replit.com if you prefer.

Step 1 — Plan First: Temperature Converter

Goal: Convert a temperature from Celsius to Fahrenheit.

Ask the group: What are the steps in plain English?

```
// Take a celsius temperature as input  
// Multiply by 9/5  
// Add 32  
// Return the result
```

Formula: $F = C \times 9/5 + 32$

Step 2 — Code It

```
function convertTemp(celsius) {  
  let fahrenheit = celsius * 9 / 5 + 32;  
  return fahrenheit;  
}  
  
console.log("0°C:", convertTemp(0));    // 32  
console.log("100°C:", convertTemp(100)); // 212  
console.log("37°C:", convertTemp(37));  // 98.6
```

Ask the group: What is 37°C in real life?

Step 3 — Plan: Reverse a String

Goal: Return a string with its characters in reverse order.

Ask the group: What are the steps?

```
// Start with an empty result string
// Loop through the input string from the last character to the first
// Add each character to the result
// Return the result
```

This is Q2 in the assignment. Try writing the pseudocode before the code.

Step 4 — Code It

```
function reverseString(str) {  
  let result = "";  
  for (let i = str.length - 1; i >= 0; i--) {  
    result += str[i];  
  }  
  return result;  
}  
  
console.log(reverseString("hello")); // "olleh"  
console.log(reverseString(""));     // ""
```

Ask the group: Why do we start at `str.length - 1`?

Step 5 — Grade Calculator Preview

Q8–Q11 build three small functions that work together.

```
// Q8: Average of an array of scores
function calculateAverage(scores) { /* ... */ }

// Q9: Letter grade from a number average
function getLetterGrade(average) { /* ... */ }

// Q10: true if passing, false otherwise
function passed(letterGrade) { /* ... */ }
```

Demo: What does `getLetterGrade(75)` return?

Extension Activity

Only if students are ready and time allows.

Extension — isPrime

Challenge: Write a function that returns `true` if a number is prime.

A prime number is only divisible by 1 and itself (e.g., 2, 3, 5, 7, 11).

```
function isPrime(num) {  
  if (num < 2) return false;  
  for (let i = 2; i <= Math.sqrt(num); i++) {  
    if (num % i === 0) return false;  
  }  
  return true;  
}
```

Try it: `isPrime(7)`, `isPrime(12)`, `isPrime(1)`, `isPrime(0)`

Assignment Preview

13 questions this week — from simple functions to a full grade calculator:

Questions	Topic
Q1–Q5	Algorithm functions (temp, reverse, tip, multiply, average)
Q6–Q7	Prime numbers (<code>isPrime</code> , <code>getPrimesUpTo</code>)
Q8–Q11	Grade calculator (four functions that work together)
Q12–Q13	Callbacks (<code>buttonPushed</code> , <code>simulateButtonPush</code>)

Also: **Create and submit your first pull request.**

Tips for the Assignment

1. **Write pseudocode first** — plan your steps before writing JavaScript
2. **Test each function on its own** — don't wait until Q11 to run anything
3. **Watch out for edge cases** — empty arrays, 0, and 1 are common traps
4. **For Q6–Q7**, check that `isPrime(0)` and `isPrime(1)` both return `false`
5. **Stuck?** Ask an AI for hints, not answers:

"Give me 3 hints for this problem without showing me the solution."

Wrap-Up

Zoom chat: Rate your confidence from 1 to 5

How comfortable do you feel breaking a problem into smaller functions?

Before next session:

- Work through the **lesson materials** (algorithms, pseudocode, callbacks, pull requests)
- Complete the **13-question assignment**
- **Create and submit** your first pull request on GitHub
- Post questions in the **Slack** `#discussion` channel anytime

Resources

Resource	What it's for
Algorithms in Everyday Life	Real-world algorithm examples
W3Schools: Callbacks	Callback reference and examples
GitHub Docs: About Pull Requests	PR documentation
List of Prime Numbers — Wikipedia	Useful for testing Q6–Q7
Slack <code>#discussion</code> channel	Questions between sessions

Next week: HTML Basics — time to build web pages!

Great work today!

You've leveled up your problem-solving skills.

Questions? Reach out on Slack anytime.